government_bond_pricing

November 4, 2026

1 Government bond pricing in OSEM

This notebook walks through how OSEM treats a government bond position: from the input file, to projected cash flows, to a calibrated market-consistent price. It is the companion of `bond_pricing.ipynb`, which covers corporate bonds. Both bond types take their cash flows, pricing and calibration from the same class, `FixedCouponBond`, so the mathematics is the same. This notebook concentrates on what is specific to government bonds and on how the two types run together.

On a very high level:

1) Each government bond is saved into a GovBond object.
2) GovBond objects go into the same BondPortfolio as the corporate bonds.
3) Based on the BondPortfolio and market conditions, the cash flows resulting from each bond are reconstructed into a matrix where the cash flow dates are in the columns and individual bonds in rows.
4) Based on the cash flows and the risk-free curve, the z-spread is calculated using the bisection algorithm to return the current market price. For a government bond, that spread is the only place where the sovereign's credit and liquidity risk enter the model.

1.1 Table of contents

1. Methodology
   - Government and corporate bonds compared
   - Notation
   - Step 1 — Cash flow dates
   - Step 2 — Date fractions
   - Step 3 — Cash flow amounts
   - Step 4 — Pricing off the risk-free curve
   - Step 5 — Calibrating the z-spread
   - Step 6 — Calibrating the whole portfolio
   - Step 7 — Use in the full OSEM run
   - What is not modelled
2. Setup
   - Environment and imports
   - Run parameters
3. The government bond portfolio
   - Loading the bonds
   - Which fields drive the price

    - Rows in the wrong file
4. Steps 1 and 3 in numbers: cash flow dates and amounts
    - Coupon and notional flows
    - Cash flow matrices
5. Step 4 in numbers: pricing off the risk-free curve
    - The risk-free curve
    - Pricing state at the modelling date
    - Risk-free price: the gap the z-spread has to close
6. Step 5 in numbers: calibrating one bond
7. Step 6 in numbers: calibrating the whole portfolio
8. Government and corporate bonds in one portfolio
9. Appendix: input data
    - notebooks.ini
    - Government_Bond_Portfolio.csv
    - Parameters.csv and the EIOPA files

1.2 1. Methodology

How OSEM turns a government bond description into a price, step by step. The steps are the ones `bond_pricing.ipynb` describes for corporate bonds, because both bond types inherit them from `FixedCouponBond`. No code in this section; the sections that follow run each step on the input portfolio.

1.2.1 Government and corporate bonds compared

	`CorpBond`	`GovBond`
Input file	`Corporate_Bond_Portfolio.csv`	`Government_Bond_Portfolio.csv`
Loader	`get_corporate_bonds`	`get_government_bonds`
`Asset_Type` in the file	`Corporate_Bond`	`Government_Bond`
Issuer field	`nace`, the sector code	`issuer_country`
Credit fields	Sector, country and stress spreads, recovery rate, default probability; loaded but not used	None
Field checks	`_validate_bond_terms`, then recovery rate and default probability	`_validate_bond_terms`, then a non-empty `issuer_country`
Cash flows, pricing, z-spread calibration	Inherited from `FixedCouponBond`	Inherited from `FixedCouponBond`
Portfolio in a run	`BondPortfolio`	The same `BondPortfolio`

`GovBond` has no credit fields. They would add nothing: for a corporate bond they are loaded but play no part in the price, since the calibrated z-spread carries all credit and liquidity risk.

1.2.2 Notation

This table defines the mathematical notation that will be used through the rest of the workbook

Symbol	Meaning
MD	modelling date
VD	valuation date, the date cash flows are discounted to ($VD = MD$ at $t = 0$)
E	end of the modelling window (`settings.end_date`)
I	issue date
M	maturity date
L	last cash flow date, $\min(E, M)$
N	notional amount
c	coupon rate, paid on **each** coupon date
f	coupon frequency (payments/year)
MV	market value (price) at the modelling date
$y(t)$	risk-free spot rate for maturity t, from the EIOPA curve
z	z-spread added to $y(t)$ at every maturity

c is the rate paid on each coupon date, not an annual rate. A bond quoted with an annual coupon c_{ann} paid f times a year is entered with `Coupon_Rate` $= c_{ann}/f$. Italian government bonds (BTPs) pay twice a year, so the 2.45% BTP maturing on 1 September 2050 (IT0005398406) is entered with `Coupon_Rate` 0.01225 and `Frequency` 2.

1.2.3 Step 1 — Cash flow dates

Every bond produces two kinds of cash flow: coupons and the return of notional. A bond still alive when the modelling window closes is redeemed there at par, so both stop at

$$L = \min(E, M)$$

Notional repayment date:

$$t_M^d = L$$

Coupon dates:

$$I < t_1^d < t_2^d < \cdots < t_k^d \leq L$$

The coupon grid is anchored on the issue date and stepped by $12/f$ months. Payments falling before MD are dropped, since only future cash flows are priced. The grid has to land on the maturity date, so in the input file I is the last coupon date on or before the bond's first issue rather than the first issue date itself. A bond first issued between two coupon dates, such as a Bund issued in January that pays every 15 February, would otherwise get its coupons on the wrong day.

Government bonds are often long; 30-year and 50-year issues are common, and Austria has issued 100-year bonds. The Slovenian bond in the input portfolio matures after E: it is repaid at par on E, and its z-spread is calibrated against those shortened cash flows.

In numbers: section 4.

1.2.4 Step 2 — Date fractions relative to the valuation date

Each remaining cash flow date is converted to a year fraction from the valuation date:

$$t_i = \frac{t_i^d - VD}{365.25}$$

At the modelling date $VD = MD$, which is the case this notebook demonstrates. In a full run the valuation date moves forward with every period and the fractions are re-measured from the new date.

Step 2 runs inside `price_bond` *and has no table of its own.*

1.2.5 Step 3 — Cash flow amounts

Notional cash flow (paid once, at L):

$$cf_M = N$$

Coupon cash flows (paid at each coupon date):

$$cf_i = N \cdot c$$

In numbers: section 4.

1.2.6 Step 4 — Pricing off the risk-free curve

Discounting the cash flows with the risk-free spot curve alone gives

$$MV_{\text{rf}} = \sum_{i=1}^{k} \frac{cf_i}{(1 + y(t_i))^{t_i}} + \frac{cf_M}{(1 + y(t_M))^{t_M}}$$

For a euro area government bond, this curve does not depend on the issuer. EIOPA derives one euro curve from swap rates, less a credit risk adjustment, and publishes it for every euro area member. The `Slovenia` curve this run uses is the euro curve, as section 5 shows, so German, French and Italian bonds are all discounted on the same $y(t)$. Everything that separates them has to come from z in Step 5.

A sovereign can trade richer than the swap curve, as German bonds did in 2023. Its market price is then above MV_{rf}, and the z-spread that closes the gap is negative.

In numbers: section 5.

1.2.7 Step 5 — Calibrating the z-spread to match the market price

OSEM finds a single constant z added to the risk-free yield at every maturity such that the discounted cash flows reproduce the bond's observed market price:

$$MV = \sum_{i=1}^{k} \frac{cf_i}{(1 + y(t_i) + z)^{t_i}} + \frac{cf_M}{(1 + y(t_M) + z)^{t_M}}$$

There is no closed-form solution for z, so OSEM finds it with a **bisection search** (`bisection_spread`) over the bracket $[-0.2, 0.2]$ that `calibrate_bond_portfolio` uses. The bracket includes negative spreads, so a bond priced above MV_{rf} calibrates the same way as one priced below it. If the market price is outside the prices at the two ends of the bracket, `bisection_spread` raises a `ValueError` naming the bond instead of returning a bracket bound. The search stops when the bracket is narrower than `precision` $= 10^{-8}$, a tolerance on the spread rather than on the price.

In numbers: section 6.

1.2.8 Step 6 — Calibrating the whole portfolio

`calibrate_bond_portfolio` repeats Step 5's bisection for every bond in the portfolio, storing one z per asset in `zspread_df`. Each bond is calibrated on its own, so a government bond gets the same z whether it is calibrated alone or together with the corporate bonds.

In numbers: section 7 and section 8.

1.2.9 Step 7 — How this is used in the full OSEM run

This notebook prices bonds once, at $t = 0$. In a full run (`main.py`):

1. `main.py` loads the corporate bond file, then the government bond file if `ALM.ini` names one (`government_bonds` in `[INPUT]`), and keys all the bonds by asset id with `bonds_by_asset_id`. Both types share the bond DataFrames, which are indexed by asset id, so an id used twice, in one file or across the two, stops the run with an error.
2. From then on there is a single `BondPortfolio`, with one set of cash flow matrices (`cpn_df`, `not_df`) and one set of `bd_price_df`, `bd_zspread_df` and `bd_units_df`. The z-spreads are calibrated **once**, at the modelling date. In every later period the bonds are repriced with that *same, fixed* z-spread on the period's projected curve, matured bonds are closed, and `trade()` buys or sells the same share of every holding, government bonds included.
3. `Results.csv` reports coupons, notionals and `Bond market value` for both types together, and `Government bond market value` and `Corporate bond market value` on their own; the two add up to `Bond market value`. The run report draws them as separate bands of the assets chart.

1.2.10 What is not modelled

- **Currency.** A government bond is assumed to be in the currency of the run's curve. A bond in another currency would need its own curve and an exchange-rate model.
- **Default.** There is no default or downgrade model, for government or corporate bonds.
- **Spread stress.** Solvency II exempts bonds of EEA governments in their own currency from spread risk. The run has no spread stress yet, so there is nothing for the exemption to act on.
- **Clean and dirty prices.** `price_bond` discounts the whole next coupon, so it produces a dirty price. Government bonds are quoted clean: enter the clean price plus accrued interest as `Market_Price`, or the z-spread absorbs the accrued interest.
- **Irregular first coupons.** Every coupon is a full period's coupon. A bond issued shortly before MD whose first coupon period is longer or shorter than the rest pays a regular coupon instead.

- **Other structures.** Only fixed-coupon bonds that repay their notional at maturity are supported, so no inflation-linked, floating-rate or callable bonds. The input file leaves them out.
- **New issues.** Matured bonds are not replaced, so the government share of the portfolio falls as the bonds mature.

1.3 2. Setup

Prepares the environment and loads the run parameters. Nothing in this section is specific to bonds.

1.3.1 Environment and imports

Locate the repository root (the folder holding `ALM.ini`), make the `osem` package importable from `src/`, and import the parts of OSEM this notebook uses.

[1]:
```
import os
import sys
import numpy as np
import pandas as pd
from dataclasses import replace
from dateutil.relativedelta import relativedelta
from pathlib import Path

# Walk up to the repository root, the folder holding ALM.ini
base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
base_folder = os.getcwd()  # Get current working directory

from osem.CurvesClass import Curves
from osem.ImportData import (get_configuration, get_settings, import_SWEiopa,
                             get_corporate_bonds, get_government_bonds)
from osem.BondClasses import BondPortfolio, GovBond, bonds_by_asset_id
from osem.ConfigurationClass import Configuration
from osem.MainLoop import create_cashflow_dataframe, holdings_market_value
```

1.3.2 Run parameters

The notebooks read `notebooks/data/notebooks.ini`, a copy of `ALM.ini` that names a small, fixed input set in `notebooks/data/Input`, so the results below do not change with the model's live `Input` folder. It gives the location of every input file, including the government bond file:

[2]:
```
conf: Configuration
conf = get_configuration(os.path.join(base_folder, "notebooks", "data",␣
 ↪"notebooks.ini"), os)
```

[3]:
```
# Relative to the repository root, which is the working directory
parameters_file = os.path.relpath(conf.input_parameters)
corporate_bond_portfolio_file = os.path.relpath(conf.
 ↪input_corporate_bond_portfolio)
government_bond_portfolio_file = os.path.relpath(
    conf.input_government_bond_portfolio)
```

The government bond file named in `notebooks.ini`:

---

[4]:
```
print(government_bond_portfolio_file)
```

```
notebooks\data\Input\Government_Bond_Portfolio.csv
```

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

[5]:
```
settings = get_settings(parameters_file)
```

The run that the rest of the notebook prices against:

---

[6]:
```
run_settings = pd.DataFrame(
    # Rendered as strings: in a mixed-type column pandas applies one float␣
 ↪format to every
    # numeric value, which would show a precision of 1e-10 as 0.0
    {"Value": [str(settings.modelling_date),
               str(settings.end_date),
               str(settings.n_proj_years),
               settings.country,
               f"{settings.precision:g}",
               f"{settings.tau:g}"]},
    index=["Modelling date",
           "End of modelling window",
           "Projection years",
           "Country",
           "Precision (curve calibration)",
           "Tau"],
)

display(run_settings)
```

	Value
Modelling date	2023-04-29
End of modelling window	2073-04-29
Projection years	50
Country	Slovenia

```
Precision (curve calibration)       1e-10
Tau                                0.0001
```

1.4 3. The government bond portfolio

1.4.1 Loading the bonds

`get_government_bonds` reads the government bond file and yields one `GovBond` per row. `bonds_by_asset_id`, the same helper `main.py` uses, keys them by asset id and raises if an id appears twice.

[7]:
```
gov_input = bonds_by_asset_id(
    get_government_bonds(government_bond_portfolio_file))
```

The input file holds four bonds, each showing one feature that shapes the cash flows:

Asset id	Bond	Feature
DE0001102580	Germany 0% 15/02/2032	Zero coupon: every coupon is 0
FR0010171975	France 4% 25/04/2055	Annual coupon
IT0005398406	Italy 2.45% 01/09/2050	Semi-annual coupon, entered at half the annual rate
SI0002104121	Slovenia 0.6875% 03/03/2081	Matures after the end of the modelling window

The Italian bond as a GovBond:

---

[8]:
```
display(gov_input["IT0005398406"])
```

```
GovBond(asset_id='IT0005398406', issuer_country='Italy', issue_date=datetime.
↪date(2019, 9, 1), maturity_date=datetime.date(2050, 9, 1), coupon_rate=0.
↪01225, notional_amount=100.0, zspread=0.0, frequency=<Frequency.BIANNUAL: 2>,␣
↪units=100.0, market_price=81.7821)
```

`BondPortfolio` wraps a dictionary of these objects, keyed by asset id, and provides the portfolio-level methods used in the rest of the notebook. It is the same class that holds the corporate bonds.

[9]:
```
gov_portfolio = BondPortfolio(gov_input)
```

The whole government portfolio at a glance. `Annual coupon` is derived, not an input: it is `Coupon rate` times `Frequency`. `Market value` is `Units` times `Market price`; its total is the first value of the `Government bond market value` column in `notebooks/data/Output/Results.csv`.

---

[10]:
```
portfolio_overview = pd.DataFrame(
    [{"Issuer": bond.issuer_country,
      "Maturity": bond.maturity_date,
      "Notional": bond.notional_amount,
      "Coupon rate": bond.coupon_rate,
      "Frequency": int(bond.frequency),
      "Annual coupon": bond.coupon_rate * int(bond.frequency),
      "Units": bond.units,
      "Market price": bond.market_price,
      "Market value": bond.units * bond.market_price}
     for bond in gov_portfolio.bonds.values()],
    index=list(gov_portfolio.bonds),
)

display(portfolio_overview)
print(f"Total market value: {portfolio_overview['Market value'].sum():,.2f}")
```

```
                Issuer    Maturity  Notional  Coupon rate  Frequency  \
DE0001102580   Germany  2032-02-15     100.0     0.000000          1
FR0010171975    France  2055-04-25     100.0     0.040000          1
IT0005398406     Italy  2050-09-01     100.0     0.012250          2
SI0002104121  Slovenia  2081-03-03     100.0     0.006875          1

              Annual coupon  Units  Market price  Market value
DE0001102580       0.000000  100.0       73.5773       7357.73
FR0010171975       0.040000  100.0      108.9074      10890.74
IT0005398406       0.024500  100.0       81.7821       8178.21
SI0002104121       0.006875  100.0       34.5930       3459.30

Total market value: 29,885.98
```

1.4.2 Which fields drive the price

Seven of the ten `GovBond` fields enter the price. The other three identify the position and size it.

`GovBond` field	CSV column	Used in pricing	Role
`asset_id`	`Asset_ID`	–	Key of the portfolio dictionary and row index of every DataFrame
`issuer_country`	`Issuer_Country`	No	Issuing government; must not be empty. It does not choose the curve: every bond is priced on the curve of `country` in `Parameters.csv`
`issue_date`	`Issue_Date`	Yes	Anchors the coupon grid (I, Step 1)
`maturity_date`	`Maturity_Date`	Yes	Notional repayment date (M, Step 1)

GovBond field	CSV column	Used in pricing	Role
`frequency`	`Frequency`	Yes	Spacing of the coupon dates, $12/f$ months (f, Step 1)
`notional_amount`	`Notional_Amount`	Yes	Notional repaid at L (N, Step 3)
`coupon_rate`	`Coupon_Rate`	Yes	Coupon paid on each coupon date, as a fraction of N (c, Step 3)
`market_price`	`Market_Price`	Yes	Calibration target (MV, Step 5)
`zspread`	`Z_Spread`	Starting value only	Seeds `zspread_df`; overwritten by the calibrated z-spread (Step 6)
`units`	`Units`	No	Number of bonds held; scales the cash flows in the main loop, not the per-unit price

1.4.3 Rows in the wrong file

Both bond loaders check the `Asset_Type` of every row. Pointed at the corporate bond file, `get_government_bonds` stops at the first row, instead of reading corporate columns as if they were government ones:

---

[11]:
```
try:
    list(get_government_bonds(corporate_bond_portfolio_file))
except ValueError as error:
    print(error)
```

```
Asset BE6285455497 in notebooks\data\Input\Corporate_Bond_Portfolio.csv has
Asset_Type Corporate_Bond, but this file can only hold Government_Bond rows
```

1.5 4. Steps 1 and 3 in numbers: cash flow dates and amounts

This section builds the coupon and notional schedules of Step 1 and the amounts of Step 3 for every government bond. Step 2, the conversion to year fractions, happens inside `price_bond` when the flows are discounted in the next section.

1.5.1 Coupon and notional flows

The coupon payments and the return of the notional are simulated separately.

`create_coupon_flows` returns the coupon dates and amounts of every bond, as a dictionary keyed by asset id:

[12]:
```
coupon_flows = gov_portfolio.create_coupon_flows(
    modelling_date=settings.modelling_date, end_date=settings.end_date)
```

`create_maturity_flows` returns the notional repayments in the same shape:

[13]:
```
notional_flows = gov_portfolio.create_maturity_flows(
    terminal_date=settings.end_date)
```

A compact view of the same two dictionaries, one row per bond. The German bond's coupons are all 0. The Italian bond pays twice a year, so it has more coupons than the French bond although it matures five years earlier, each at half its annual rate. The Slovenian bond outlives the modelling window: its notional is repaid on E, and its coupons stop at the last coupon date before it.

---

[14]:
```
coupon_schedule = pd.DataFrame(
    [{"Issuer": gov_input[aid].issuer_country,
      "Coupons": len(coupons),
      "Coupon amount": next(iter(coupons.values())),
      "First coupon": min(coupons),
      "Last coupon": max(coupons),
      "Notional repaid": min(notional_flows[aid])}
     for aid, coupons in coupon_flows.items()],
    index=list(coupon_flows),
)

display(coupon_schedule)
```

```
                Issuer  Coupons  Coupon amount First coupon Last coupon  \
DE0001102580   Germany        9         0.0000   2024-02-15  2032-02-15
FR0010171975    France       32         4.0000   2024-04-25  2055-04-25
IT0005398406     Italy       55         1.2250   2023-09-01  2050-09-01
SI0002104121  Slovenia       50         0.6875   2024-03-03  2073-03-03

              Notional repaid
DE0001102580       2032-02-15
FR0010171975       2055-04-25
IT0005398406       2050-09-01
SI0002104121       2073-04-29
```

1.5.2 Cash flow matrices

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all bonds as rows. The non-zero entries are the cash flow per unit of the bond at that date. The columns are the unique payment dates across the portfolio, from `unique_dates_profile`, computed separately for coupons and notional repayments:

[15]:
```
unique_list = gov_portfolio.unique_dates_profile(coupon_flows)
unique_terminal_list = gov_portfolio.unique_dates_profile(notional_flows)
```

A dataframe of cash flows and notional amount payments is created:

[16]:
```
# Dataframe with government bond coupon cash flows
cash_flows = create_cashflow_dataframe(coupon_flows, unique_list)
# Dataframe with government bond notional cash flows
notional_cash_flows = create_cashflow_dataframe(notional_flows,
                                                unique_terminal_list)
```

The coupon matrix has one row per bond and one column per payment date. Its size, then the coupons paid in the first year of the run; dates on which no coupon is paid are left out:

---

[17]:
```
print(f"Coupon matrix: {cash_flows.shape[0]} bonds by {cash_flows.shape[1]}␣
 ↪payment dates")

first_year = [d for d in cash_flows.columns
              if d < settings.modelling_date + relativedelta(years=1)]
first_year_coupons = cash_flows[first_year]
display(first_year_coupons.loc[:, (first_year_coupons != 0).any()])
```

```
Coupon matrix: 4 bonds by 146 payment dates
```

	2023-09-01	2024-03-01	2024-03-03	2024-04-25
DE0001102580	0.000	0.000	0.0000	0.0
FR0010171975	0.000	0.000	0.0000	4.0
IT0005398406	1.225	1.225	0.0000	0.0
SI0002104121	0.000	0.000	0.6875	0.0

Cash flow dataframe with notional amount payments and dates. The Slovenian bond is repaid on E, not on its maturity date:

---

[18]:
```
display(notional_cash_flows)
```

	2032-02-15	2050-09-01	2055-04-25	2073-04-29
DE0001102580	100.0	0.0	0.0	0.0
FR0010171975	0.0	0.0	100.0	0.0
IT0005398406	0.0	100.0	0.0	0.0
SI0002104121	0.0	0.0	0.0	100.0

1.6 5. Step 4 in numbers: pricing off the risk-free curve

This section prices the cash flows from section 4 off the risk-free curve (Step 4).

1.6.1 The risk-free curve

`import_SWEiopa()` reads the yield curve of the run's `country` from the EIOPA files, and the `Curves` object calibrates and projects it, exactly as in `main.py`.

[19]:
```
[maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(
    settings.EIOPA_param_file, settings.EIOPA_curves_file, settings.country)
# Curves object with information about term structure
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,
                settings.modelling_date, settings.country)
```

[20]:
```
curves.set_observed_term_structure(
    maturity_vec=np.asarray(curve_country.index, dtype=np.float64),
    yield_vec=np.asarray(curve_country, dtype=np.float64))
curves.calc_fwd_rates()
# The +1 matches main.py: the last projection period still has to be priced, so␣
 ↪the
# projected curve has to reach one year past the end of the modelling window.
curves.project_forward_rate(settings.n_proj_years+1)
curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)
```

The run's curve is the euro curve. Reading the `Euro` column of the same file and comparing it with the run's curve at a few maturities, and then at every maturity:

---

[21]:
```
_, euro_curve, _, _ = import_SWEiopa(
    settings.EIOPA_param_file, settings.EIOPA_curves_file, "Euro")

curve_comparison = pd.DataFrame({settings.country: curve_country,
                                 "Euro": euro_curve})
display(curve_comparison.iloc[[0, 4, 9, 19, 29]])
print(f"The {settings.country} curve equals the euro curve at every maturity: "
      f"{bool((curve_country == euro_curve).all())}")
```

Country	Slovenia	Euro
1	0.03271	0.03271
5	0.03488	0.03488
10	0.03577	0.03577
20	0.03597	0.03597
30	0.03514	0.03514

```
The Slovenia curve equals the euro curve at every maturity: True
```

1.6.2 Pricing state at the modelling date

`init_bond_portfolio_to_dataframe` sets up the three DataFrames the pricing works on, each with one column for the modelling date: the market price of each bond, its z-spread and the units held.

[22]:
```
[market_price_df, zspread_df, units_df] = (
    gov_portfolio.init_bond_portfolio_to_dataframe(settings.modelling_date))
```

`zspread_df` starts from the `Z_Spread` column of the government bond file, which is 0 for every bond. The calibration in sections 6 and 7 overwrites it:

---

[23]:
```
display(zspread_df)
```

	2023-04-29
DE0001102580	0.0
FR0010171975	0.0
IT0005398406	0.0
SI0002104121	0.0

All pricing in this notebook happens at the modelling date, so the projection year is 0:

[24]:
```
proj_period = 0
```

1.6.3 Risk-free price: the gap the z-spread has to close

Pricing the cash flows at $z = 0$ gives MV_{rf} of Step 4. A bond priced below it is cheaper than the euro curve implies and needs a positive z-spread; a bond priced above it needs a negative one. The input file sets every market price to MV_{rf} itself, rounded to four decimals, so there is no gap to close and every z-spread will come out at 0. Observed market prices would instead give each bond its spread over the euro curve:

---

[25]:
```
risk_free_price = {}
for asset_id, bond in gov_portfolio.bonds.items():
    risk_free_price[asset_id] = bond.price_bond(
        cash_flows.loc[asset_id], notional_cash_flows.loc[asset_id],
        settings.modelling_date, proj_period, curves, 0.0)

issuer = pd.Series({aid: b.issuer_country for aid, b in gov_input.items()})
market_price = pd.Series({aid: b.market_price for aid, b in gov_input.items()})
price_comparison = pd.DataFrame({
    "Issuer": issuer,
    "Risk free price": pd.Series(risk_free_price),
    "Market price": market_price,
})
price_comparison["Gap to close"] = (price_comparison["Risk free price"]
                                    - price_comparison["Market price"])

display(price_comparison)
```

	Issuer	Risk free price	Market price	Gap to close
DE0001102580	Germany	73.577270	73.5773	-0.000030
FR0010171975	France	108.907407	108.9074	0.000007

```
IT0005398406     Italy        81.782123       81.7821      0.000023
SI0002104121  Slovenia        34.593003       34.5930      0.000003
```

1.7 6. Step 5 in numbers: calibrating one bond

Step 5 for the Italian bond, IT0005398406. At its file price the bisection would simply return 0. To show it finding a spread, this section calibrates the bond as if it traded 5 below its file price, as it would if the market asked for a premium over the euro curve. The lower price needs a positive spread, and the bracket $[-0.2, 0.2]$ includes it. The bond at the lower price is a copy made with `replace`, so the portfolio is unchanged.

[26]:
```
italian_bond = gov_portfolio.bonds["IT0005398406"]
cheaper_bond = replace(italian_bond, market_price=italian_bond.market_price - 5)

calibrated_spread = cheaper_bond.bisection_spread(x_start=-0.2
                                , x_end=0.2
                                , modelling_date=settings.modelling_date
                                , end_date=settings.end_date
                                , proj_period=proj_period
                                , curves=curves
                                , precision=0.00000001
                                , max_iter=100000)
```

The calibrated z-spread, as a decimal and in basis points:

---

[27]:
```
print(f"{calibrated_spread:.6f} ({calibrated_spread * 10000:.1f} bp)")
```

```
0.003457 (34.6 bp)
```

Discounting the bond's cash flows at the calibrated z-spread gives back the lower market price:

[28]:
```
calibrated_bond = cheaper_bond.price_bond(
    cash_flows.loc["IT0005398406"], notional_cash_flows.loc["IT0005398406"],
    settings.modelling_date, proj_period, curves, calibrated_spread)
```

---

[29]:
```
print(f"Repriced value {calibrated_bond:.6f}, "
      f"market price {cheaper_bond.market_price:.4f}")
```

```
Repriced value 76.782106, market price 76.7821
```

1.8 7. Step 6 in numbers: calibrating the whole portfolio

`calibrate_bond_portfolio` repeats the bisection for every bond (Step 6) and writes the results into `zspread_df`:

[30]:
```
zspread_df = gov_portfolio.calibrate_bond_portfolio(zspread_df, settings,
                                                    proj_period, curves)
```

---

[31]:
```
display(zspread_df)
```

```
                2023-04-29
DE0001102580 -5.364418e-08
FR0010171975  5.960464e-09
IT0005398406  1.788139e-08
SI0002104121  5.960464e-09
```

Repricing the portfolio with the calibrated spreads reproduces the market prices. The residual is the price cost of stopping the bisection once the spread bracket is narrower than `precision`, as described in Step 5.

Every calibrated z-spread is 0, to within that precision, because the market prices in the input file are the risk-free prices of section 5. With observed prices instead, the spreads would follow the euro government bond market: German bonds below the swap-based euro curve, Italian bonds furthest above it.

---

[32]:
```
calibrated_price_df = gov_portfolio.price_bond_portfolio(
    cash_flows, notional_cash_flows, settings, proj_period, curves,
    zspread_df, market_price_df.copy(), settings.modelling_date)

calibration_check = pd.DataFrame({
    "Issuer": issuer,
    "Calibrated z-spread": zspread_df.iloc[:, 0],
    "Z-spread (bp)": (zspread_df.iloc[:, 0] * 10000).round(1),
    "Repriced value": calibrated_price_df.iloc[:, 0],
    "Market price": market_price,
})
calibration_check["Residual"] = (calibration_check["Repriced value"]
                                 - calibration_check["Market price"])

display(calibration_check)
```

```
                Issuer  Calibrated z-spread  Z-spread (bp)  Repriced value  \
DE0001102580   Germany        -5.364418e-08           -0.0       73.577303
FR0010171975    France         5.960464e-09            0.0      108.907395
IT0005398406     Italy         1.788139e-08            0.0       81.782096
SI0002104121  Slovenia         5.960464e-09            0.0       34.592996
```

```
              Market price  Residual
DE0001102580       73.5773  0.000003
FR0010171975      108.9074 -0.000005
IT0005398406       81.7821 -0.000004
SI0002104121       34.5930 -0.000004
```

1.9 8. Government and corporate bonds in one portfolio

`main.py` keeps no separate government portfolio. It loads both files and keys every bond by asset id in one dictionary, as Step 7 describes, so each step above runs once for both types:

[33]:
```
bonds = list(get_corporate_bonds(corporate_bond_portfolio_file))
bonds += get_government_bonds(government_bond_portfolio_file)
bond_portfolio = BondPortfolio(bonds_by_asset_id(bonds))
gov_ids = bond_portfolio.asset_ids_of(GovBond)

[all_price_df, all_zspread_df, all_units_df] = (
    bond_portfolio.init_bond_portfolio_to_dataframe(settings.modelling_date))
all_zspread_df = bond_portfolio.calibrate_bond_portfolio(
    all_zspread_df, settings, proj_period, curves)
```

The combined portfolio by bond type. The government bonds are just over a third of the bond holdings by market value. Their z-spreads are all 0, while the corporate ones range from about 0.05% to 1.5%:

---

[34]:
```
bond_type = pd.Series({aid: type(bond).__name__
                       for aid, bond in bond_portfolio.bonds.items()})
market_value = all_units_df.iloc[:, 0] * all_price_df.iloc[:, 0]
all_spreads = all_zspread_df.iloc[:, 0]

by_type = pd.DataFrame({
    "Bonds": bond_type.value_counts(),
    "Market value": market_value.groupby(bond_type).sum(),
    "Lowest z-spread": all_spreads.groupby(bond_type).min(),
    "Highest z-spread": all_spreads.groupby(bond_type).max(),
})

display(by_type)
```

```
          Bonds  Market value  Lowest z-spread  Highest z-spread
CorpBond      5      52017.82     4.620969e-04      1.466404e-02
GovBond       4      29885.98    -5.364418e-08      1.788139e-08
```

Each bond is calibrated on its own, so the government bonds keep the spreads of section 7.

Their market value, taken from the shared DataFrames at the ids `asset_ids_of(GovBond)` lists, as `main.py` does, is the opening value of `Government bond market value` in `notebooks/data/Output/Results.csv`:

---

[35]:
```
same_spreads = bool((all_zspread_df.loc[gov_ids] == zspread_df).all().all())
government_value = holdings_market_value(all_price_df.loc[gov_ids],
                                         all_units_df.loc[gov_ids],
                                         settings.modelling_date)

print(f"Same z-spreads as when calibrated alone: {same_spreads}")
print(f"Government bond market value at the modelling date: "
      f"{government_value:,.2f}")
```

```
Same z-spreads as when calibrated alone: True
Government bond market value at the modelling date: 29,885.98
```

Because the two types share these DataFrames, an asset id may be used only once across both files. A government bond given the id of a corporate bond stops the run:

---

[36]:
```
clash = replace(gov_input["SI0002104121"], asset_id=bonds[0].asset_id)
try:
    bonds_by_asset_id([*bonds, clash])
except ValueError as error:
    print(error)
```

```
Asset ID BE6285455497 is used by two bonds (CorpBond and GovBond). Bond asset
IDs must be unique across the corporate and government bond input files.
```

1.10 9. Appendix: input data

The input files this notebook reads, all in `notebooks/data/Input`. Their locations come from the `[INPUT]` section of `notebooks/data/notebooks.ini`, except the EIOPA files, whose paths are set in `Parameters.csv`. Dates are written as `dd/mm/yyyy`.

1.10.1 notebooks.ini

The notebooks' copy of `ALM.ini`, with the same sections and keys. Two keys in the `[INPUT]` section name the bond files:

Key	Example	Meaning
`corporate_bonds`	`Corporate_Bond_Portfolio.csv`	Corporate bond file, in the input folder. An `ALM.ini` written before government bonds were added calls this key `bonds`
`government_bonds`	`Government_Bond_Portfolio.csv`	Government bond file, in the input folder. Optional: without it a run holds no government bonds, `Government bond market value` is 0, and an `ALM.ini` written before government bonds were added still loads

1.10.2 Government_Bond_Portfolio.csv

The government bond portfolio, one row per bond position. Loaded by `get_government_bonds`, one `GovBond` per row. Section 3 shows which of these fields enter the price. The four bonds are real euro area government bonds, with the terms and prices of the model's own government bond file: `Market_Price` is the price at a zero z-spread on the run's curve, so every bond calibrates to a z-spread of 0. The notebook holds 100 units of each. The examples are the Italian bond's row.

Column	Example	Meaning
`Asset_ID`	`IT0005398406`	Unique id of the position, read as text: the bond's ISIN. No corporate bond may use it either
`Asset_Type`	`Government_Bond`	Asset type; `get_government_bonds` raises if it is not `Government_Bond`
`Issuer_Country`	`Italy`	Issuing government; must not be empty
`Issue_Date`	`01/09/2019`	Issue date I; anchors the coupon grid. In this file it is the last coupon date on or before the bond's first issue, so the grid lands on the maturity date
`Maturity_Date`	`01/09/2050`	Maturity date M; the notional is repaid on it
`Notional_Amount`	`100`	Notional amount N
`Coupon_Rate`	`0.01225`	Coupon c paid on **each** coupon date, as a fraction of the notional. Not annualised: this bond pays 2.45% a year in two coupons
`Z_Spread`	`0`	Starting z-spread; replaced by the calibrated value
`Frequency`	`2`	Coupon payments per year f: 1, 2, 3, 4 or 12
`Units`	`100`	Number of bonds held
`Market_Price`	`81.7821`	Price of one bond at the modelling date, on the same scale as `Notional_Amount`, including accrued interest; the calibration target MV

1.10.3 Parameters.csv and the EIOPA files

`Parameters.csv`, `Curves_no_VA.csv` and `Param_no_VA.csv` are read exactly as for corporate bonds; the appendix of `bond_pricing.ipynb` describes their columns. Of the parameters, this notebook uses `Modelling_Date`, `n_proj_years`, `country`, `Precision`, `Tau` and the paths of the two EIOPA files.